23AIE303 - Database Management Systems — Project Report

Cloud-Native Distributed NewSQL Database with Raft Consensus and MVCC Transactions

Batch BATCH_NO · Group GROUP_NO

GitHub repository: https://github.com/sriram270506/Cloud-Native-Distributed-SQL-DB

Contents

1. Project Title
2. Team Members
3. Introduction and Problem Statement
4. Conceptual and Logical Database Design
5. Normalization and Functional Dependencies
6. Advanced SQL Implementation and Query Design
7. ER Diagram and UML Diagram
8. Relational Schema
9. Database Tables
10. Conclusion and Future Scope
11. Appendix A: Testing and Validation
12. Appendix B: Setup and Demonstration

1. Project Title

Cloud-Native Distributed NewSQL Database with Raft Consensus and MVCC Transactions

The project has two layers:

- **A distributed database engine.** Every node is a separate process. Writes are replicated with the Raft consensus protocol and run as MVCC transactions. The engine is written in Go and deploys on Docker Compose or Kubernetes.
- **A supermarket order-management database.** It is designed, normalized and queried in standard SQL, and it runs on that engine.

GitHub repository: https://github.com/sriram270506/Cloud-Native-Distributed-SQL-DB

2. Team Members

Batch: BATCH_NO Group: GROUP_NO

#	Name	Roll number
1	Member 1 Name	Roll No.
2	Member 2 Name	Roll No.
3	Member 3 Name	Roll No.

3. Introduction and Problem Statement

3.1 Introduction

A traditional relational database runs on one server. If that server fails, the data is unavailable and recent writes may be lost. NewSQL systems such as Google Spanner, CockroachDB and TiDB keep the SQL interface and ACID transactions, but spread the data across several machines. They agree on every change with a consensus protocol.

This project builds a small database of that kind from first principles:

- **Raft consensus** elects a leader and replicates each write to a majority of nodes before it is acknowledged.
- **A write-ahead log** plus a durable Raft log let a crashed node recover its state.
- **MVCC** (multi-version concurrency control) gives transactions snapshot isolation.
- **A SQL layer** supports DDL, DML, joins, grouping and aggregates.
- **An HTTP API and a React console** let users run SQL against any node.

3.2 Problem statement

A supermarket needs one consistent record of its customers, addresses, products, orders, order lines and payments. The record must:

1. stay available and correct when a server fails, with no lost acknowledged writes and no split-brain;
2. apply multi-statement updates atomically, for example creating an order with its lines and payment;
3. enforce data integrity through keys and constraints, and stay free of update anomalies through normalization;
4. answer analytical questions such as revenue per category, top customers and unsold stock with SQL;
5. accept client input safely, through parameterized queries, so SQL injection is not possible.

3.3 Objectives

- Design a normalized (3NF/BCNF) relational schema with full PK, FK, UNIQUE, CHECK and ON DELETE CASCADE constraints.
- Implement a replicated SQL database: Raft over TCP, a durable BoltDB log, linearizable reads and automatic failover.
- Implement MVCC transactions and all-or-nothing batches that are replicated in the same order on every node.
- Demonstrate advanced SQL (multi-table joins, correlated and nested subqueries, GROUP BY/HAVING, window functions, views) and validate everything with automated tests.

4. Conceptual and Logical Database Design

4.1 Conceptual design: entities and relationships

Entity	Description	Key attribute
Customer	A registered shopper	customer_id
Address	A postal address owned by a customer (home, work, billing or shipping)	address_id
Product	An item in the catalogue, with price and stock	product_id
Order	One purchase by one customer on a date	order_id
Order Item	One product line within an order (associative entity)	order_item_id
Payment	A payment made against an order	payment_id

Relationship	Cardinality	Participation
Customer owns Address	1 : N	An address must belong to a customer (total); a customer may have none (partial)
Customer places Order	1 : N	Every order has exactly one customer
Order contains Product (through Order Item)	M : N, resolved to two 1 : N	An order has at least one line; a product may be on no order
Order is settled by Payment	1 : N	A payment belongs to exactly one order

4.2 Logical design: from ER model to relations

- Each strong entity becomes a relation, and its key becomes the primary key.
- Each 1:N relationship becomes a foreign key on the N side: `addresses.customer_id`, `orders.customer_id` and `payments.order_id`.
- The M:N Order–Product relationship becomes the associative relation `order_items`. It has two foreign keys and a `UNIQUE(order_id, product_id)` constraint, so a product appears at most once per order.
- Domains are enforced with CHECK constraints, for example status and payment-method lists, positive prices and quantities, and `updated_at ≥ created_at`.
- Referential actions follow ownership:
    - Deleting a customer cascades to their addresses and orders, and from orders on to order lines and payments.
    - Deleting a product that has been sold is blocked (`ON DELETE RESTRICT`), so sales history is kept.

4.3 System design: how the database stores and replicates the data

1. A client sends SQL to any node over HTTP. A follower forwards the request to the current leader.
2. The leader binds the `?` parameters into literals and appends the statement batch to the Raft log as one entry.
3. The entry is replicated over TCP and becomes committed once a majority of nodes has saved it to its BoltDB log.
4. Every node applies committed entries in log order through its state machine. The state machine runs the batch in the SQL engine and rolls back the whole batch if any statement fails.

5. The SQL engine stores schema and rows through MVCC into a key-value store. That store fsyncs a write-ahead log before each change.
6. Reads are served by the leader after it confirms its leadership with a quorum, so every read sees all acknowledged writes (linearizable).

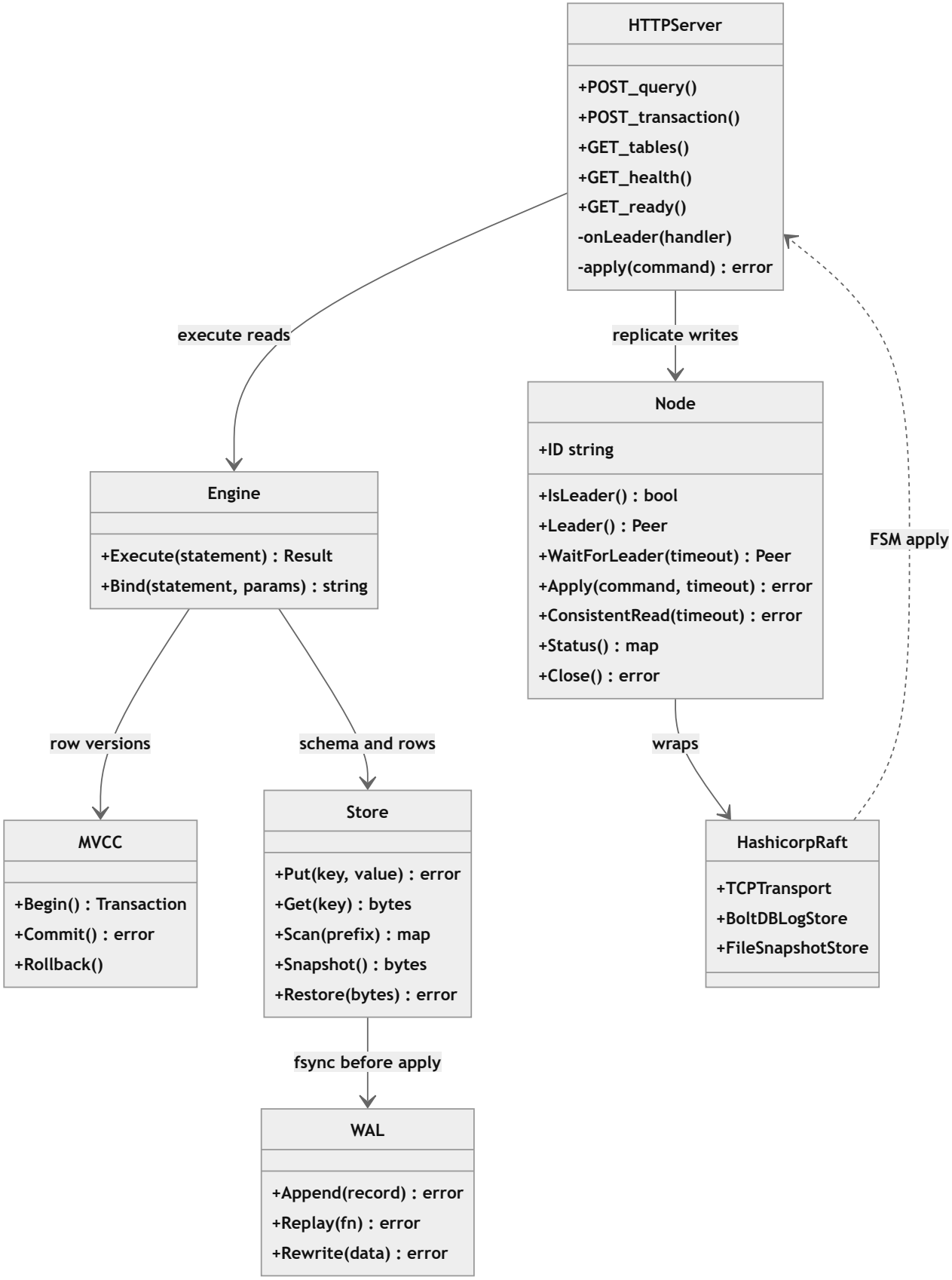


Figure 4.1: System components (Go packages)

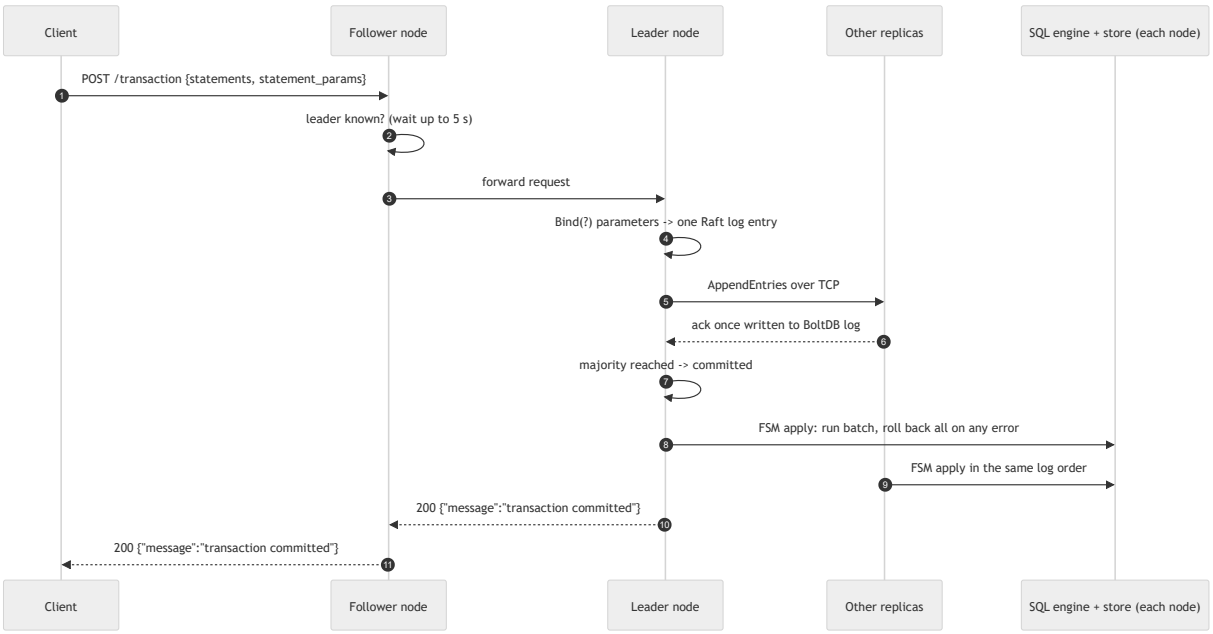


Figure 4.2: Write path of a replicated transaction

5. Normalization and Functional Dependencies

5.1 Starting point: an unnormalized order sheet

Without design, a store might keep one wide row per order:

```
ORDER_SHEET(order_id, order_date, customer_name, customer_email, customer_city,
            {product_name, category, unit_price, quantity}*, payment_method, amount)
```

This sheet has three problems:

- **Repeating group.** The product list is repeated inside one row, which violates 1NF.
- **Redundant data.** Customer and product details are repeated on every order.
- **Anomalies.**
  - Update: changing a product's price means editing many rows.
  - Insertion: a new product cannot be stored until someone orders it.
  - Deletion: deleting a customer's only order also deletes the customer.

5.2 Functional dependencies

Relation	Functional dependencies	Candidate keys
customers	customer_id → first_name, last_name, email, created_at, updated_at, status; email → customer_id	{customer_id}, {email}
addresses	address_id → customer_id, line1, line2, city, state, postal_code, country, type; (customer_id, type) → address_id	{address_id}, {customer_id, type}
products	product_id → product_name, category, unit_price, stock_qty, created_at, updated_at; product_name → product_id	{product_id}, {product_name}
orders	order_id → customer_id, order_date, total_amount, status, created_at, updated_at	{order_id}
order_items	order_item_id → order_id, product_id, quantity, unit_price, line_total, created_at; (order_id, product_id) → order_item_id	{order_item_id}, {order_id, product_id}
payments	payment_id → order_id, method, amount, paid_at, status, transaction_ref; transaction_ref → payment_id	{payment_id}, {transaction_ref}

5.3 Normal forms

- **1NF.** Every attribute is atomic. The repeating product group became separate `order_items` rows, and addresses became their own relation.
- **2NF.** Every relation has a single-attribute primary key, so partial dependencies on part of a composite key cannot occur. In `order_items`, the alternate key (order_id, product_id) determines the whole row. Product attributes such as name and category are not stored there, because they depend only on product_id. They live in `products`.
- **3NF.** No non-key attribute depends on another non-key attribute. Customer data is not copied into orders, and payment data is not copied into orders.
- **BCNF.** In every relation, the determinant of every non-trivial FD (listed above) is a candidate key. So every relation is in BCNF.

5.4 Controlled redundancy

Three stored values are deliberate redundancies:

- **`order_items.unit_price`** is the historical price at the time of sale. It is not the current `products.unit_price`, so it is a separate fact and not redundant.
- **`order_items.line_total`** is derived. It is kept consistent by `CHECK (line_total = ROUND(quantity * unit_price, 2))`.
- **`orders.total_amount`** is derived from the order's lines. It is stored for fast reporting, and query Q9 checks it against the lines (it returns 0 rows for the sample data).

6. Advanced SQL Implementation and Query Design

6.1 DDL implementation

The full DDL is in `database/schema.sql`:

- six tables with PRIMARY KEY, FOREIGN KEY (CASCADE and RESTRICT), UNIQUE, NOT NULL, DEFAULT and named CHECK constraints;
- six secondary indexes;
- the `order_summary` view.

The script is portable SQL, tested on SQLite 3.45.1 and written to run on PostgreSQL.

Sample data is in `database/seed.sql`: customers 10, addresses 10, products 10, orders 10, order_items 12, payments 10 rows. It is loaded in one transaction.

6.2 Queries and their outputs

The fourteen queries below are documented in `database/queries.sql`. Their outputs were produced by running them on the seeded database.

Q1. Multi-table join (4 tables): every order line with customer, product and order date

a full invoice view; shows how normalized tables are recombined with joins.

```
SELECT o.order_id,
       c.first_name || ' ' || c.last_name AS customer,
       o.order_date,
       p.product_name,
       oi.quantity,
       oi.line_total
FROM orders o
JOIN customers   c  ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products    p  ON p.product_id  = oi.product_id
ORDER BY o.order_id, p.product_name;
```

Output (12 rows)

order_id	customer	order_date	product_name	quantity	line_total
1	Emma Johnson	2024-06-01	Accessory 1	2	9.98
1	Emma Johnson	2024-06-01	Widget A	1	19.99
2	Liam Smith	2024-06-02	Widget B	1	29.99
3	Olivia Williams	2024-06-03	Gadget X	1	9.99
4	Noah Brown	2024-06-04	Accessory 1	1	4.99
4	Noah Brown	2024-06-04	Device One	1	199.99
5	Ava Jones	2024-06-05	Gadget Y	1	14.99
6	William Garcia	2024-06-06	Tool Alpha	2	99.98
7	Sophia Miller	2024-06-07	Accessory 2	1	6.99
8	James Davis	2024-06-08	Device One	1	199.99
9	Isabella Martinez	2024-06-09	Device Two	1	299.99

order_id	customer	order_date	product_name	quantity	line_total
10	Logan Hernandez	2024-06-10	Tool Alpha	1	49.99

Q2. GROUP BY with aggregates: revenue, units and distinct orders per category

category-level sales report for the store manager.

```
SELECT p.category,
       COUNT(DISTINCT oi.order_id) AS orders,
       SUM(oi.quantity)            AS units_sold,
       ROUND(SUM(oi.line_total), 2) AS revenue,
       ROUND(AVG(oi.unit_price), 2) AS avg_unit_price
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

Output (5 rows)

category	orders	units_sold	revenue	avg_unit_price
devices	3	3	699.97	233.32
tools	2	3	149.97	49.99
widgets	2	2	49.98	24.99
gadgets	2	2	24.98	12.49
accessories	3	4	21.96	5.66

Q3. GROUP BY ... HAVING: categories whose revenue exceeds 100

HAVING filters groups after aggregation, unlike WHERE which filters rows.

```
SELECT p.category, ROUND(SUM(oi.line_total), 2) AS revenue
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
HAVING SUM(oi.line_total) > 100
ORDER BY revenue DESC;
```

Output (2 rows)

category	revenue
devices	699.97
tools	149.97

Q4. Correlated subquery: customers who spent more than the average order value

the inner query is evaluated against the outer customer row.

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer,
       (SELECT ROUND(SUM(o.total_amount), 2) FROM orders o WHERE o.customer_id = c.customer_id) AS
total_spent
FROM customers c
WHERE (SELECT SUM(o.total_amount) FROM orders o WHERE o.customer_id = c.customer_id)
      > (SELECT AVG(total_amount) FROM orders)
ORDER BY total_spent DESC;
```

Output (4 rows)

customer_id	customer	total_spent
9	Isabella Martinez	299.99
4	Noah Brown	204.98
8	James Davis	199.99
6	William Garcia	99.98

Q5. NOT EXISTS subquery: products that have never been ordered

anti-join to find dead stock.

```
SELECT p.product_id, p.product_name, p.category, p.stock_qty
FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id)
ORDER BY p.product_id;
```

Output (1 rows)

product_id	product_name	category	stock_qty
6	Tool Beta	tools	40

Q6. IN subquery: customers who paid at least once with PayPal

a nested query across three tables without an explicit join.

```
SELECT customer_id, first_name, last_name, email
FROM customers
WHERE customer_id IN (
  SELECT o.customer_id
  FROM orders o
  WHERE o.order_id IN (SELECT order_id FROM payments WHERE method = 'paypal')
)
ORDER BY customer_id;
```

Output (2 rows)

customer_id	first_name	last_name	email
3	Olivia	Williams	olivia.williams@example.com
7	Sophia	Miller	sophia.miller@example.com

Q7. LEFT JOIN + aggregate: order count and spend for every customer, including those with none

outer join keeps unmatched rows; COALESCE turns NULL into 0.

```
SELECT c.customer_id,
       c.first_name,
       COUNT(o.order_id)                    AS orders,
       COALESCE(ROUND(SUM(o.total_amount), 2), 0) AS spent
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY spent DESC, c.customer_id;
```

Output (10 rows)

customer_id	first_name	orders	spent
9	Isabella	1	299.99
4	Noah	1	204.98
8	James	1	199.99
6	William	1	99.98
10	Logan	1	49.99
2	Liam	1	29.99
1	Emma	1	29.97
5	Ava	1	14.99
3	Olivia	1	9.99
7	Sophia	1	6.99

Q8. Common table expression + window function: rank products by revenue within each category

RANK() OVER (PARTITION BY ...) computes a per-group ranking without collapsing rows.

```
WITH product_revenue AS (
  SELECT p.category, p.product_name, COALESCE(SUM(oi.line_total), 0) AS revenue
  FROM products p
  LEFT JOIN order_items oi ON oi.product_id = p.product_id
  GROUP BY p.category, p.product_name
)
SELECT category,
       product_name,
       ROUND(revenue, 2) AS revenue,
       RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rank_in_category
FROM product_revenue
ORDER BY category, rank_in_category;
```

Output (10 rows)

category	product_name	revenue	rank_in_category
accessories	Accessory 1	14.97	1
accessories	Accessory 2	6.99	2
devices	Device One	399.98	1
devices	Device Two	299.99	2
gadgets	Gadget Y	14.99	1
gadgets	Gadget X	9.99	2
tools	Tool Alpha	149.97	1
tools	Tool Beta	0.0	2
widgets	Widget B	29.99	1
widgets	Widget A	19.99	2

Q9. Data-integrity check: orders whose stored total differs from the sum of their lines

total_amount is a stored derived value; this query proves it is consistent (expect 0 rows).

```
SELECT o.order_id, o.total_amount, ROUND(SUM(oi.line_total), 2) AS computed_total
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.total_amount
HAVING ROUND(SUM(oi.line_total), 2) <> o.total_amount;
```

Output (0 rows)

No rows (expected).

Q10. CASE expression: classify customers into spending tiers

conditional logic inside SELECT for segmentation.

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer,
       ROUND(SUM(o.total_amount), 2) AS spent,
       CASE
         WHEN SUM(o.total_amount) >= 200 THEN 'gold'
         WHEN SUM(o.total_amount) >= 50  THEN 'silver'
         ELSE 'bronze'
       END AS tier
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY spent DESC;
```

Output (10 rows)

customer_id	customer	spent	tier
9	Isabella Martinez	299.99	gold
4	Noah Brown	204.98	gold
8	James Davis	199.99	silver
6	William Garcia	99.98	silver
10	Logan Hernandez	49.99	bronze
2	Liam Smith	29.99	bronze
1	Emma Johnson	29.97	bronze
5	Ava Jones	14.99	bronze
3	Olivia Williams	9.99	bronze
7	Sophia Miller	6.99	bronze

Q11. Scalar subquery in SELECT: each product's price compared to its category average

per-row comparison against a group statistic.

```
SELECT p.product_name,
       p.category,
       p.unit_price,
       ROUND((SELECT AVG(p2.unit_price) FROM products p2 WHERE p2.category = p.category), 2) AS
category_avg,
       ROUND(p.unit_price - (SELECT AVG(p2.unit_price) FROM products p2 WHERE p2.category = p.category),
2) AS diff_from_avg
FROM products p
ORDER BY p.category, p.unit_price DESC;
```

Output (10 rows)

product_name	category	unit_price	category_avg	diff_from_avg
Accessory 2	accessories	6.99	5.99	1.0
Accessory 1	accessories	4.99	5.99	-1.0
Device Two	devices	299.99	249.99	50.0
Device One	devices	199.99	249.99	-50.0
Gadget Y	gadgets	14.99	12.49	2.5
Gadget X	gadgets	9.99	12.49	-2.5
Tool Beta	tools	59.99	54.99	5.0
Tool Alpha	tools	49.99	54.99	-5.0
Widget B	widgets	29.99	24.99	5.0
Widget A	widgets	19.99	24.99	-5.0

Q12. View usage: orders that are fully paid, read through the order_summary view

views hide join/subquery complexity behind a virtual table.

```
SELECT order_id, customer_name, total_amount, amount_paid, item_lines
FROM order_summary
WHERE amount_paid >= total_amount
ORDER BY order_id;
```

Output (10 rows)

order_id	customer_name	total_amount	amount_paid	item_lines
1	Emma Johnson	29.97	29.97	2
2	Liam Smith	29.99	29.99	1
3	Olivia Williams	9.99	9.99	1
4	Noah Brown	204.98	204.98	2
5	Ava Jones	14.99	14.99	1
6	William Garcia	99.98	99.98	1
7	Sophia Miller	6.99	6.99	1
8	James Davis	199.99	199.99	1
9	Isabella Martinez	299.99	299.99	1
10	Logan Hernandez	49.99	49.99	1

Q13. UNION: one contact list of cities from home and work addresses, labelled by type

combine two result sets with the same shape.

```
SELECT city, state, 'home' AS address_type FROM addresses WHERE type = 'home'
UNION
SELECT city, state, 'work' AS address_type FROM addresses WHERE type = 'work'
ORDER BY address_type, city;
```

Output (10 rows)

city	state	address_type
Austin	TX	home
Columbus	OH	home

city	state	address_type
Denver	CO	home
Madison	WI	home
Orlando	FL	home
Portland	OR	home
Raleigh	NC	home
Springfield	IL	home
San Diego	CA	work
Seattle	WA	work

Q14. Date filtering + aggregates: daily revenue in the first week of June 2024

range predicate on a DATE column with BETWEEN.

```
SELECT order_date, COUNT(*) AS orders, ROUND(SUM(total_amount), 2) AS revenue
FROM orders
WHERE order_date BETWEEN '2024-06-01' AND '2024-06-07'
GROUP BY order_date
ORDER BY order_date;
```

Output (7 rows)

order_date	orders	revenue
2024-06-01	1	29.97
2024-06-02	1	29.99
2024-06-03	1	9.99
2024-06-04	1	204.98
2024-06-05	1	14.99
2024-06-06	1	99.98
2024-06-07	1	6.99

6.3 SQL on the distributed engine

The Raft-replicated engine supports:

- `CREATE TABLE`, `INSERT`, `UPDATE`, `DELETE` and `DROP TABLE`;
- `SELECT` with `WHERE` (AND, OR, LIKE and comparisons), a two-table inner equi-`JOIN`, `GROUP BY`, `HAVING`, `COUNT/SUM/AVG/MIN/MAX`, `ORDER BY`, `LIMIT` and `OFFSET`;
- multi-statement transactions through `/transaction`.

Client values are always sent as bound parameters (`?` placeholders with a `params` array). The engine hex-encodes every string literal before parsing, so a value can never be read as SQL. Section 6.4 shows an injection attempt returning 0 rows.

6.4 Live cluster output

Generated by python database/seed_cluster.py against a live three-node Raft cluster.

Node node-1 reports state **Follower**, leader node-2, term 2, commit index 7.

Point lookup with a bound parameter

```
SELECT customer_id, first_name, last_name, email FROM customers WHERE email = ?
```

Parameters: ["ava.jones@example.com"]

customer_id	first_name	last_name	email
5	Ava	Jones	ava.jones@example.com

Injection attempt is treated as data (0 rows)

```
SELECT * FROM customers WHERE email = ?
```

Parameters: ["x' OR '1'='1"]

No rows.

Join: orders with customer names

```
SELECT orders.order_id, customers.first_name, orders.total_amount FROM orders JOIN customers ON
orders.customer_id = customers.customer_id ORDER BY order_id
```

orders.order_id	customers.first_name	orders.total_amount
1	Emma	29.97
2	Liam	29.99
3	Olivia	9.99
4	Noah	204.98
5	Ava	14.99
6	William	99.98
7	Sophia	6.99
8	James	199.99
9	Isabella	299.99
10	Logan	49.99

GROUP BY with COUNT and AVG

```
SELECT category, COUNT(*) AS products, AVG(unit_price) AS avg_price FROM products GROUP BY category ORDER
BY category
```

category	products	avg_price
accessories	2	5.99
devices	2	249.99
gadgets	2	12.49
tools	2	54.99
widgets	2	24.99

GROUP BY ... HAVING

```
SELECT method, COUNT(*) AS payments, SUM(amount) AS total FROM payments GROUP BY method HAVING payments >
1
```

method	payments	total
paypal	2	16.98
card	8	929.88

Filter with OR, ORDER BY and LIMIT

```
SELECT product_name, unit_price, stock_qty FROM products WHERE category = ? OR stock_qty < ? ORDER BY unit_price DESC LIMIT 5
```

Parameters: ["tools", 30]

product_name	unit_price	stock_qty
Device Two	299.99	10
Device One	199.99	20
Tool Beta	59.99	40
Tool Alpha	49.99	50

7. ER Diagram and UML Diagram

7.1 ER diagram

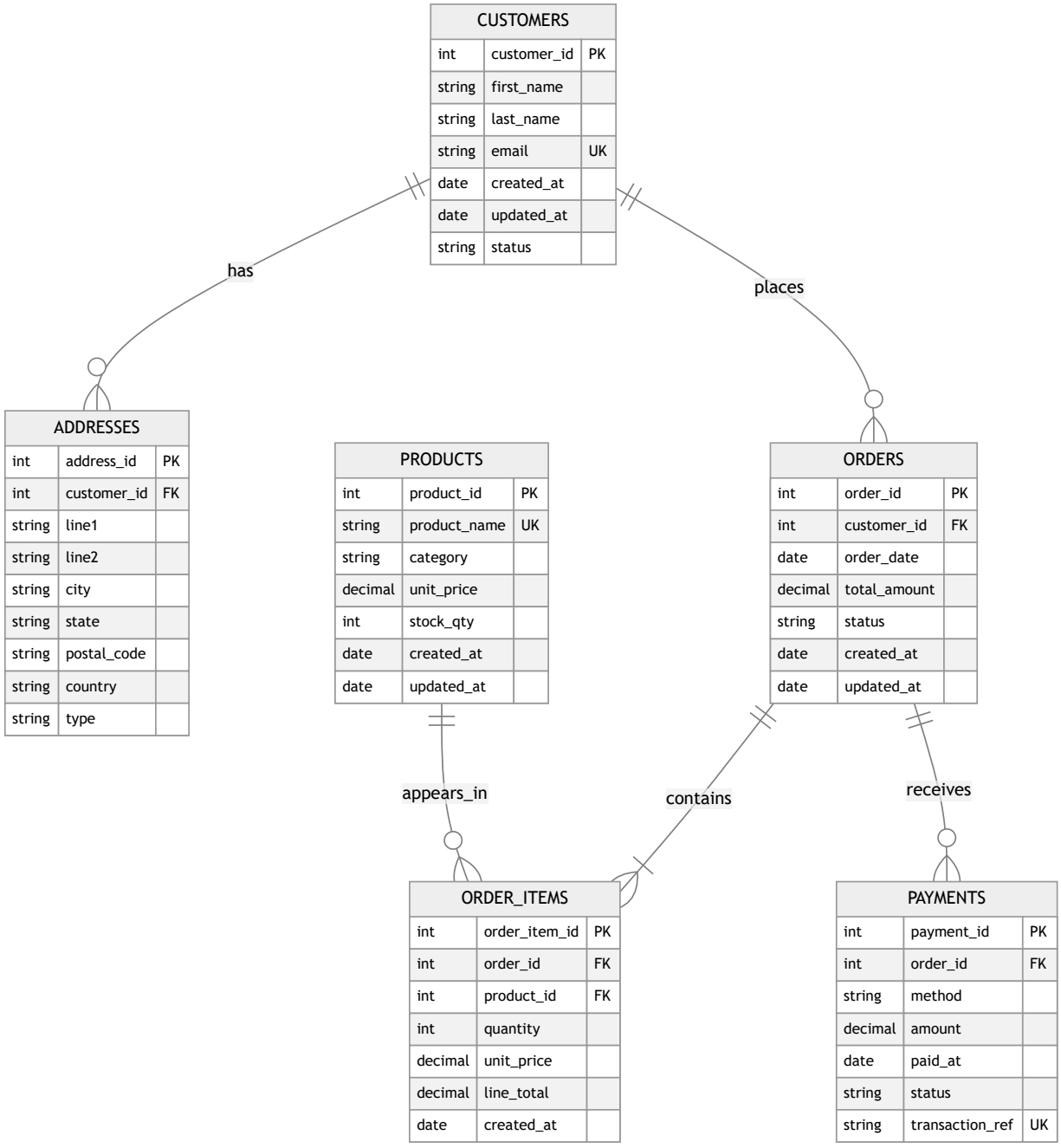


Figure 7.1: Entity-relationship diagram (PK = primary key, FK = foreign key, UK = unique key)

7.2 UML class diagram

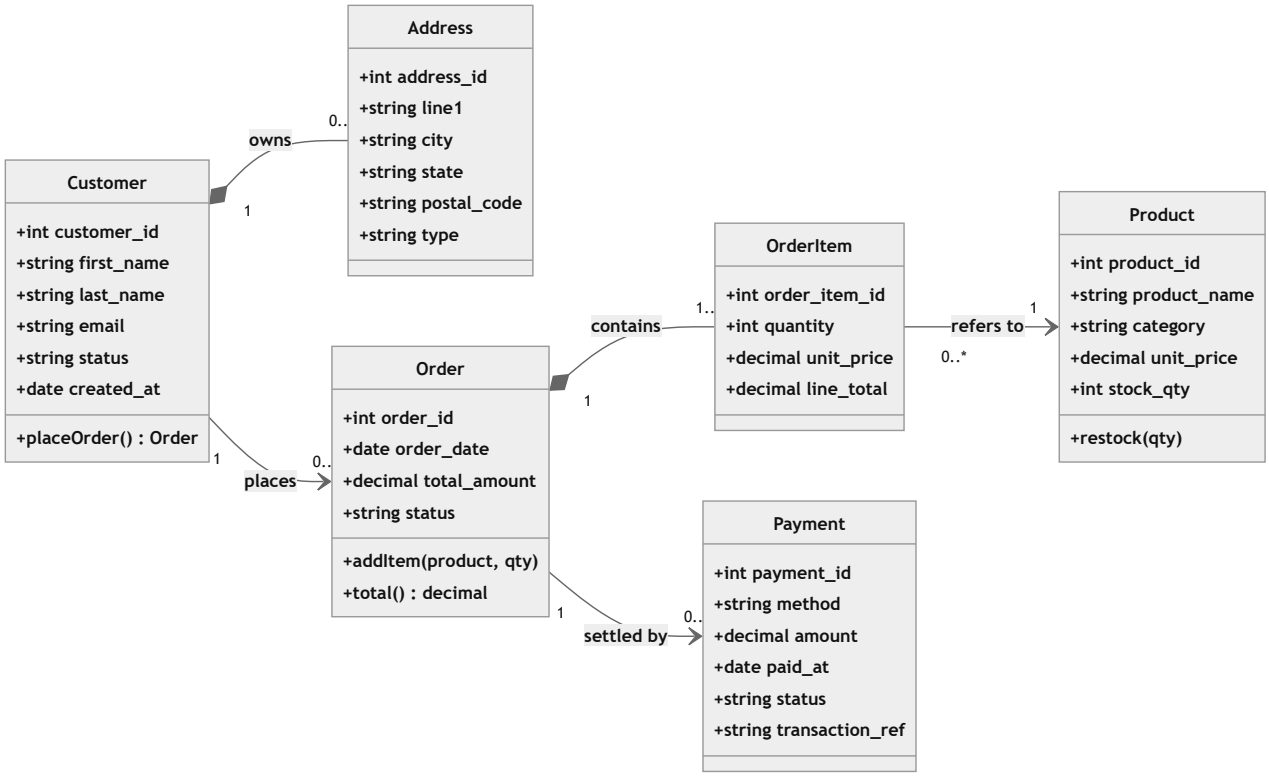


Figure 7.2: UML class diagram of the supermarket domain (composition shows ownership with cascading delete)

8. Relational Schema

Notation: underline = primary key, *italic* = foreign key.

```
CUSTOMERS  (customer_id, first_name, last_name, email UNIQUE, created_at, updated_at, status)
ADDRESSES  (address_id, customer_id → CUSTOMERS, line1, line2, city, state, postal_code, country, type)
             UNIQUE (customer_id, type)
PRODUCTS   (product_id, product_name UNIQUE, category, unit_price, stock_qty, created_at, updated_at)
ORDERS     (order_id, customer_id → CUSTOMERS, order_date, total_amount, status, created_at, updated_at)
ORDER_ITEMS(order_item_id, order_id → ORDERS, product_id → PRODUCTS, quantity, unit_price, line_total,
created_at)
             UNIQUE (order_id, product_id)
PAYMENTS   (payment_id, order_id → ORDERS, method, amount, paid_at, status, transaction_ref UNIQUE)
```

Foreign key	References	On delete	Reason
addresses.customer_id	customers.customer_id	CASCADE	Addresses exist only for their customer
orders.customer_id	customers.customer_id	CASCADE	Removing a customer removes their order history
order_items.order_id	orders.order_id	CASCADE	Lines are part of the order
order_items.product_id	products.product_id	RESTRICT	A product that was sold cannot be deleted
payments.order_id	orders.order_id	CASCADE	Payments belong to the order

9. Database Tables

Table structures, attributes, keys and constraints below are read back from the created database. Sample records come from `data/sample_db`.

9.1 customers

Attribute	Type	Null	Default	Keys
customer_id	INTEGER	NOT NULL		PK
first_name	VARCHAR(50)	NOT NULL		
last_name	VARCHAR(50)	NOT NULL		
email	VARCHAR(120)	NOT NULL		UNIQUE
created_at	DATE	NOT NULL		
updated_at	DATE	NOT NULL		
status	VARCHAR(10)	NOT NULL	'active'	

Table-level constraints

```
CONSTRAINT chk_customer_email   CHECK (email LIKE '%_@_%._%')
CONSTRAINT chk_customer_status  CHECK (status IN ('active', 'inactive', 'suspended'))
CONSTRAINT chk_customer_dates   CHECK (updated_at >= created_at)
```

Sample records (10 rows)

customer_id	first_name	last_name	email	created_at	updated_at	status
1	Emma	Johnson	emma.johnson@example.com	2024-01-10	2024-06-01	active
2	Liam	Smith	liam.smith@example.com	2024-02-12	2024-06-02	active
3	Olivia	Williams	olivia.williams@example.com	2024-03-05	2024-06-03	active
4	Noah	Brown	noah.brown@example.com	2024-03-20	2024-06-04	active
5	Ava	Jones	ava.jones@example.com	2024-04-01	2024-06-05	active
6	William	Garcia	william.garcia@example.com	2024-04-15	2024-06-06	active
7	Sophia	Miller	sophia.miller@example.com	2024-05-02	2024-06-07	active
8	James	Davis	james.davis@example.com	2024-05-10	2024-06-08	active
9	Isabella	Martinez	isabella.martinez@example.com	2024-05-20	2024-06-09	active
10	Logan	Hernandez	logan.hernandez@example.com	2024-06-01	2024-06-10	active

9.2 addresses

Attribute	Type	Null	Default	Keys
address_id	INTEGER	NOT NULL		PK
customer_id	INTEGER	NOT NULL		FK → customers(customer_id) ON DELETE CASCADE
line1	VARCHAR(100)	NOT NULL		
line2	VARCHAR(100)	NULL		
city	VARCHAR(60)	NOT NULL		
state	CHAR(2)	NOT NULL		
postal_code	VARCHAR(10)	NOT NULL		

Attribute	Type	Null	Default	Keys
country	VARCHAR(60)	NOT NULL	'USA'	
type	VARCHAR(10)	NOT NULL		

Table-level constraints

```
CONSTRAINT chk_address_type    CHECK (type IN ('home', 'work', 'billing', 'shipping'))
CONSTRAINT uq_address_per_type UNIQUE (customer_id, type)
```

Sample records (10 rows)

address_id	customer_id	line1	line2	city	state	postal_code	country	type
1	1	123 Maple St		Springfield	IL	62701	USA	home
2	2	456 Oak Ave		Madison	WI	53703	USA	home
3	3	789 Pine Rd		Columbus	OH	43085	USA	home
4	4	12 Birch Ln		Austin	TX	73301	USA	home
5	5	34 Cedar Blvd		Denver	CO	80014	USA	home
6	6	56 Elm St		Seattle	WA	98101	USA	work
7	7	78 Aspen Way		Portland	OR	97201	USA	home
8	8	90 Walnut Dr		San Diego	CA	92101	USA	work
9	9	101 Cherry Ct		Raleigh	NC	27601	USA	home
10	10	202 Poplar Pl		Orlando	FL	32801	USA	home

9.3 products

Attribute	Type	Null	Default	Keys
product_id	INTEGER	NOT NULL		PK
product_name	VARCHAR(100)	NOT NULL		UNIQUE
category	VARCHAR(40)	NOT NULL		
unit_price	NUMERIC(10, 2)	NOT NULL		
stock_qty	INTEGER	NOT NULL	0	
created_at	DATE	NOT NULL		
updated_at	DATE	NOT NULL		

Table-level constraints

```
CONSTRAINT chk_product_price CHECK (unit_price > 0)
CONSTRAINT chk_product_stock CHECK (stock_qty >= 0)
CONSTRAINT chk_product_dates CHECK (updated_at >= created_at)
```

Sample records (10 rows)

product_id	product_name	category	unit_price	stock_qty	created_at	updated_at
1	Widget A	widgets	19.99	100	2024-01-01	2024-06-01
2	Widget B	widgets	29.99	80	2024-01-05	2024-06-02
3	Gadget X	gadgets	9.99	200	2024-02-01	2024-06-03
4	Gadget Y	gadgets	14.99	150	2024-02-10	2024-06-04
5	Tool Alpha	tools	49.99	50	2024-03-01	2024-06-05
6	Tool Beta	tools	59.99	40	2024-03-15	2024-06-06

product_id	product_name	category	unit_price	stock_qty	created_at	updated_at
7	Accessory 1	accessories	4.99	500	2024-04-01	2024-06-07
8	Accessory 2	accessories	6.99	400	2024-04-10	2024-06-08
9	Device One	devices	199.99	20	2024-05-01	2024-06-09
10	Device Two	devices	299.99	10	2024-05-15	2024-06-10

9.4 orders

Attribute	Type	Null	Default	Keys
order_id	INTEGER	NOT NULL		PK
customer_id	INTEGER	NOT NULL		FK → customers(customer_id) ON DELETE CASCADE
order_date	DATE	NOT NULL		
total_amount	NUMERIC(10, 2)	NOT NULL	0	
status	VARCHAR(10)	NOT NULL	'pending'	
created_at	DATE	NOT NULL		
updated_at	DATE	NOT NULL		

Table-level constraints

```
CONSTRAINT chk_order_total   CHECK (total_amount >= 0)
CONSTRAINT chk_order_status  CHECK (status IN ('pending', 'paid', 'shipped', 'completed', 'cancelled'))
CONSTRAINT chk_order_dates   CHECK (updated_at >= created_at)
```

Sample records (10 rows)

order_id	customer_id	order_date	total_amount	status	created_at	updated_at
1	1	2024-06-01	29.97	completed	2024-06-01	2024-06-02
2	2	2024-06-02	29.99	completed	2024-06-02	2024-06-03
3	3	2024-06-03	9.99	completed	2024-06-03	2024-06-04
4	4	2024-06-04	204.98	completed	2024-06-04	2024-06-05
5	5	2024-06-05	14.99	completed	2024-06-05	2024-06-06
6	6	2024-06-06	99.98	completed	2024-06-06	2024-06-07
7	7	2024-06-07	6.99	completed	2024-06-07	2024-06-08
8	8	2024-06-08	199.99	completed	2024-06-08	2024-06-09
9	9	2024-06-09	299.99	completed	2024-06-09	2024-06-10
10	10	2024-06-10	49.99	completed	2024-06-10	2024-06-11

9.5 order_items

Attribute	Type	Null	Default	Keys
order_item_id	INTEGER	NOT NULL		PK
order_id	INTEGER	NOT NULL		FK → orders(order_id) ON DELETE CASCADE
product_id	INTEGER	NOT NULL		FK → products(product_id) ON DELETE RESTRICT
quantity	INTEGER	NOT NULL		
unit_price	NUMERIC(10, 2)	NOT NULL		
line_total	NUMERIC(10, 2)	NOT NULL		

Attribute	Type	Null	Default	Keys
created_at	DATE	NOT NULL		

Table-level constraints

```
CONSTRAINT chk_item_quantity CHECK (quantity > 0)
CONSTRAINT chk_item_price    CHECK (unit_price > 0)
CONSTRAINT chk_item_total    CHECK (line_total = ROUND(quantity * unit_price, 2))
CONSTRAINT uq_item_product_per_order UNIQUE (order_id, product_id)
```

Sample records (12 rows)

order_item_id	order_id	product_id	quantity	unit_price	line_total	created_at
1	1	1	1	19.99	19.99	2024-06-01
2	1	7	2	4.99	9.98	2024-06-01
3	2	2	1	29.99	29.99	2024-06-02
4	3	3	1	9.99	9.99	2024-06-03
5	4	9	1	199.99	199.99	2024-06-04
6	4	7	1	4.99	4.99	2024-06-04
7	5	4	1	14.99	14.99	2024-06-05
8	6	5	2	49.99	99.98	2024-06-06
9	7	8	1	6.99	6.99	2024-06-07
10	9	10	1	299.99	299.99	2024-06-09
11	8	9	1	199.99	199.99	2024-06-08
12	10	5	1	49.99	49.99	2024-06-10

9.6 payments

Attribute	Type	Null	Default	Keys
payment_id	INTEGER	NOT NULL		PK
order_id	INTEGER	NOT NULL		FK → orders(order_id) ON DELETE CASCADE
method	VARCHAR(15)	NOT NULL		
amount	NUMERIC(10, 2)	NOT NULL		
paid_at	DATE	NOT NULL		
status	VARCHAR(10)	NOT NULL	'pending'	
transaction_ref	VARCHAR(30)	NOT NULL		UNIQUE

Table-level constraints

```
CONSTRAINT chk_payment_method CHECK (method IN ('card', 'paypal', 'upi', 'cash', 'bank_transfer'))
CONSTRAINT chk_payment_amount CHECK (amount > 0)
CONSTRAINT chk_payment_status CHECK (status IN ('pending', 'paid', 'failed', 'refunded'))
```

Sample records (10 rows)

payment_id	order_id	method	amount	paid_at	status	transaction_ref
1	1	card	29.97	2024-06-01	paid	tx1001
2	2	card	29.99	2024-06-02	paid	tx1002
3	3	paypal	9.99	2024-06-03	paid	tx1003

payment_id	order_id	method	amount	paid_at	status	transaction_ref
4	4	card	204.98	2024-06-04	paid	tx1004
5	5	card	14.99	2024-06-05	paid	tx1005
6	6	card	99.98	2024-06-06	paid	tx1006
7	7	paypal	6.99	2024-06-07	paid	tx1007
8	8	card	199.99	2024-06-08	paid	tx1008
9	9	card	299.99	2024-06-09	paid	tx1009
10	10	card	49.99	2024-06-10	paid	tx1010

10. Conclusion and Future Scope

10.1 Conclusion

The project meets its objectives:

- **Relational design.** The supermarket database is normalized to BCNF and fully constrained. Automated tests check every class of constraint.
- **SQL.** Fourteen documented queries cover multi-table joins, correlated, nested and EXISTS subqueries, GROUP BY/HAVING, window functions, a CTE, a view, CASE and UNION.
- **Distributed engine.** The same data runs on a database we built ourselves. Every node is an independent process. Writes commit only on a Raft majority, and transactions are all-or-nothing on every replica. Reads are linearizable. Client input is always parameterized.
- **Fault tolerance.** In the failover demo (Appendix A), killing the leader caused an automatic election in under 2 seconds. A write sent at that moment still committed, and the restarted node caught up from its own durable log.

10.2 Future scope

- Enforce FOREIGN KEY, UNIQUE and CHECK constraints inside the distributed engine. Today they are enforced in the reference SQL schema.
- Support subqueries and joins of more than two tables in the distributed engine, using a cost-based planner.
- Split data into ranges, each with its own Raft group (multi-Raft sharding), so write throughput scales beyond one leader.
- Allow nodes to join and leave a running cluster.
- Add TLS between nodes and clients, plus authentication and role-based authorization.
- Store data in an on-disk LSM storage engine with typed columns and secondary indexes.

Appendix A: Testing and Validation

Suite	Command	What it proves
Go unit and integration tests	go test ./...	MVCC snapshot isolation and conflicts; SQL CRUD, filters, grouping, joins; parameter binding blocks injection; the WAL stays correct after rollback; a 3-node Raft cluster over TCP replicates, fails over and recovers from its log
Schema tests (16)	python -m unittest discover -s database -v	Seed counts; no dangling FKs; PK, UNIQUE, FK, RESTRICT and 8 CHECK rules reject bad rows; ON DELETE CASCADE removes dependents; every documented query runs and returns the expected rows
Cluster smoke test	scripts/smoke.ps1 or scripts/smoke.sh	Waits for a leader, then runs DDL, DML, a transaction and a read through the HTTP API
Cluster seed and demo	python database/seed_cluster.py	Loads all six tables through parameterized transactions and records the query outputs

A.1 Failover demonstration

Recorded on a local three-process cluster (scripts/cluster.ps1) after database/seed_cluster.py.

1. Healthy cluster

node	state	leader	term	commit index	applied index
node-1	Follower	node-2	2	8	8
node-2	Leader	node-2	2	8	8
node-3	Follower	node-2	2	8	8

2. Leader process node-2 killed (no graceful shutdown)

Write sent to node-3 immediately after the kill: {"message":"committed"} (returned after 1747 ms, including the new election).

node	state	leader	term	commit index	applied index
node-1	Follower	node-3	3	10	10
port 8081	down				
node-3	Leader	node-3	3	10	10

3. node-2 restarted and caught up from its Raft log

node	state	leader	term	commit index	applied index
node-1	Follower	node-3	3	10	10
node-2	Follower	node-3	3	10	10
node-3	Leader	node-3	3	10	10

Read through the restarted node:

```
{"columns":["customer_id","first_name","last_name"],"rows":
[{"customer_id":"11","first_name":"Ada","last_name":"Lovelace"}]}
```

A.2 Screenshots

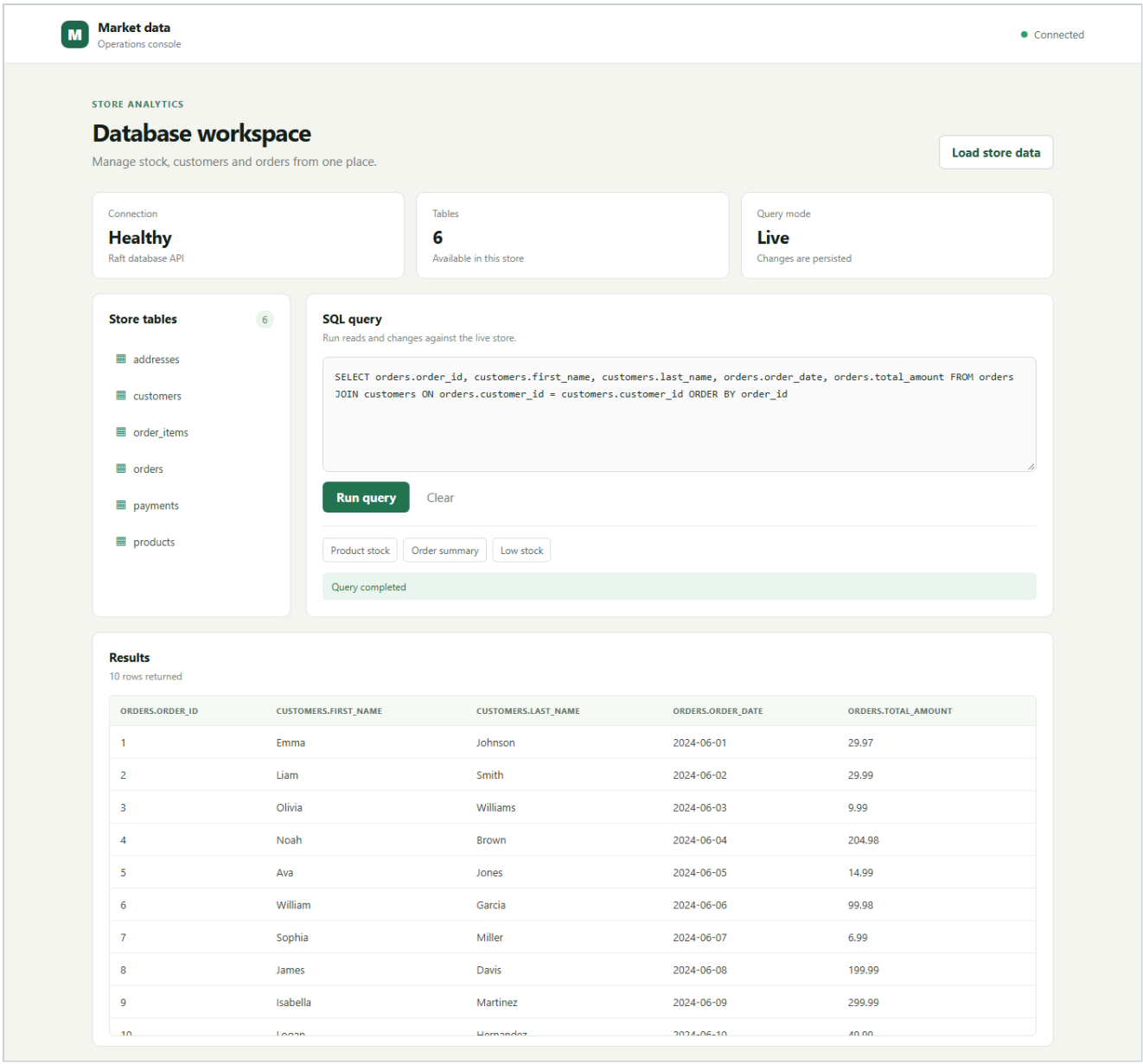


Figure A.1: React console running a join on the live cluster

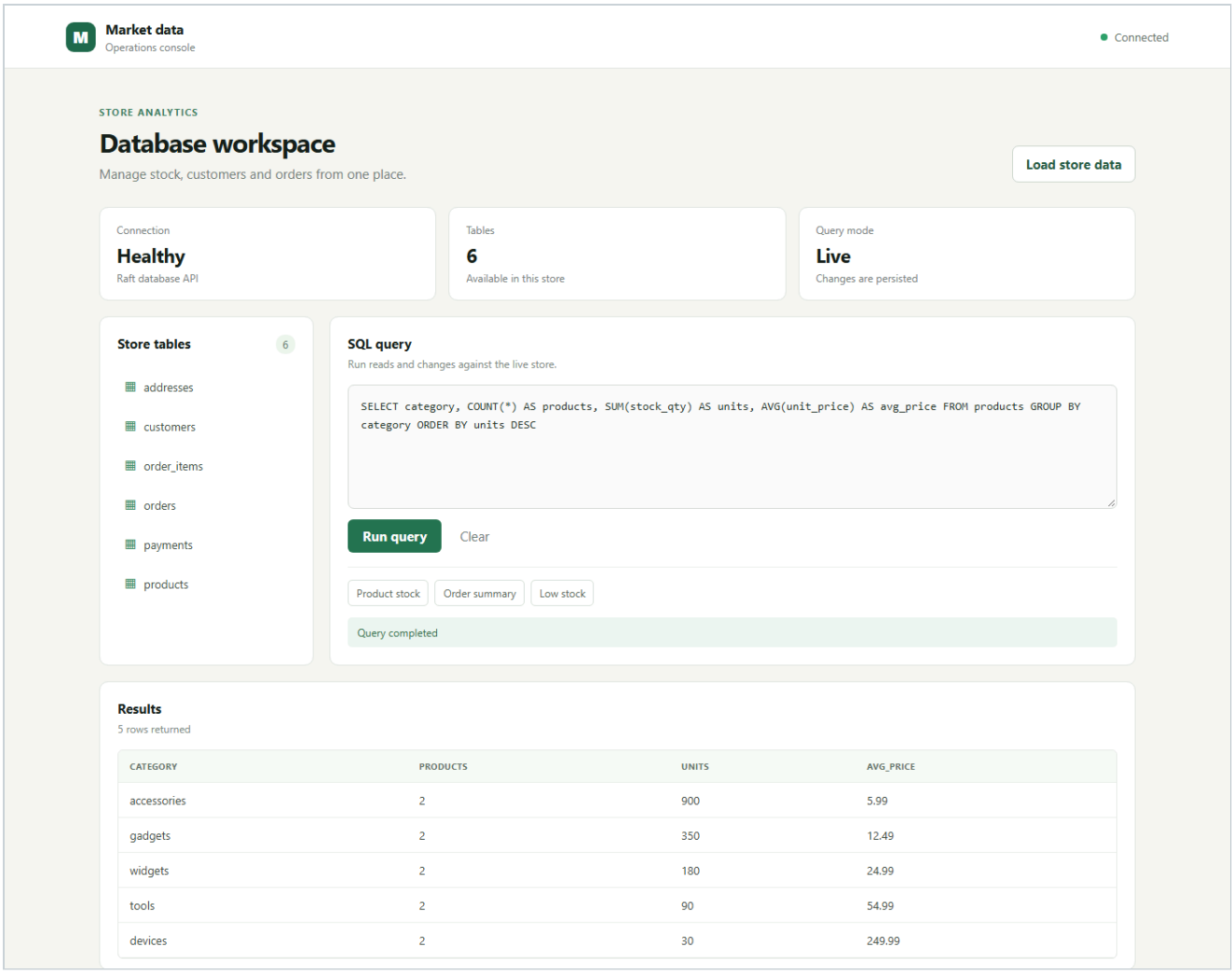


Figure A.2: GROUP BY with aggregates on the live cluster

Appendix B: Setup and Demonstration

1. Install Go 1.26 or newer, Python 3.10 or newer and Node.js 18 or newer. Docker Desktop is optional.
2. Clone the repository: `git clone https://github.com/sriram270506/Cloud-Native-Distributed-SQL-DB`
3. Run the tests: `go test ./...` and `python -m unittest discover -s database -v`
4. Build the reference database and record query outputs: `python database/run_queries.py --db database/supermarket.db`
5. Start a three-node cluster with `scripts/cluster.ps1` (Windows), `scripts/cluster.sh` (Linux/macOS) or `docker compose up -d --build`.
6. Create and populate the tables on the cluster: `python database/seed_cluster.py`
7. Start the UI: `cd frontend && npm ci && npm run dev`, then open `http://localhost:5173`
8. Demonstrate failover. Find the leader with `GET /health`, stop that process or container, then send a write to another node. It still commits. Restart the stopped node and confirm it catches up.